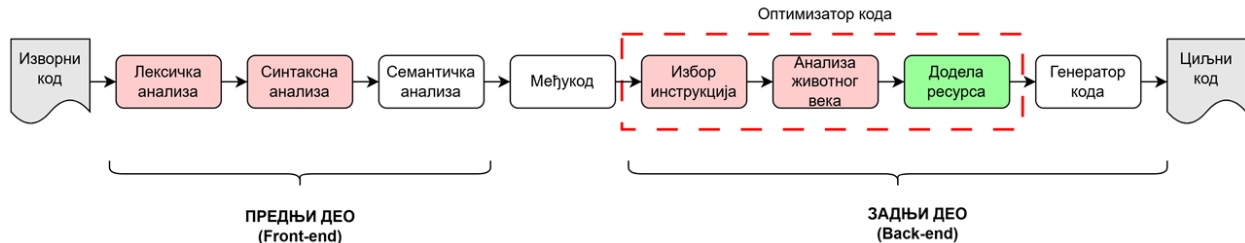


ДЕЛОВИ КОМПАЈЛЕРА

Додела ресурса

Фаза доделе ресурса представља један од најважнијих корака у задњем делу компајлера. Њен циљ је да „бесконачан број“ регистара који су постојали у међукоду сведе на реалан, ограничен број регистара које пружа циљна архитектура. Поред алокације регистара, ова фаза често укључује и оптимизацију, као што је елиминација сувишних MOVE инструкција.



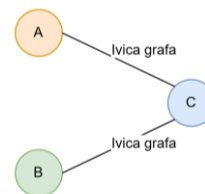
Слика 1: Основне фазе компајлера (преводноца)

Ако су две променљиве истовремено потребне програму (живе у истом тренутку), онда не могу делити исти ресурс тј. регистар. Проблем који онемогућава да се исти регистар додели двома привременим променљивама назива се **сметња**. Најчешћи узрок сметње је преклапање опсега животног века променљивих (али постоје и други разлози: на пример, ако променљива А треба да буде генерисана инструкцијом која не може да адресира регистар R тада постоји сметња између променљиве А и регистра R).

Две променљиве су у сметњи ако се њихов животни век преклапа

Граф сметњи је неусмерен граф који описује односе између променљивих у програму у контексту доделе регистара и садржи:

- **чворове графа** – променљиве,
- **ивице графа** – повезују две променљиве које су живе у истом тренутку.



Граф сметњи се често у рачунару представља **матрицом сметњи** која се користи при додели физичких регистара за променљиве. Колоне и редови матрице су исти и представљају све променљиве у програму. На пресеку одговарајуће колоне и реда биће уписан индикатор да ли постоји сметња између променљиве у колони и променљиве у реду. **Уколико сметња постоји између две променљиве, значи да им не може бити додељен исти ресурс и да су истовремено живе у неком тренутку.**

Да би се из резултата анализе животног века променљивих направио граф сметњи, користе се следећа правила:

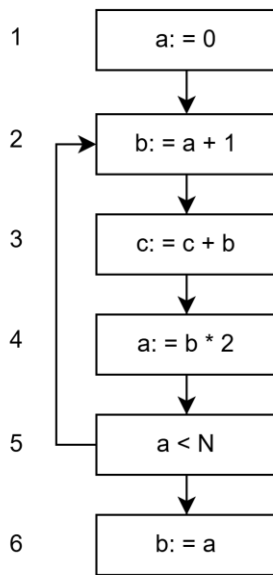
1. За сваку **дефиницију** променљиве А, у чвору који **није MOVE**, додаје се сметња између променљиве А и сваке променљиве B_i која живи на излазу чвора: $(A, B_1) \dots (A, B_n)$.
2. За сваку **MOVE** инструкцију $A = C$, додаје се сметња између променљиве А и сваке променљиве B_i живе на излазу чвора, где је $B_i \neq C$.

На Слици 2 је дат граф тока управљања једноставног програма. Анализом животног века променљивих добијамо резултате који су приказани у Табели 1. На основу резултата добијених из анализе животног века променљивих правимо граф сметњи, примењујући претходно наведена правила, Табела 2.

Неопходно је дефинисати следеће скупове за сваки чвор тј. инструкцију:

- **USE** представља променљиве које инструкција користи,
- **DEF** представља променљиве које инструкција дефинише,
- **IN** представља променљиве живе пре инструкције,
- **OUT** представља променљиве живе након инструкције.

Чворови 1, 2, 3 и 4 дефинишу променљиве А, В и С и ниједна инструкција није **MOVE**, стога се примењује 1. правило тј. додају се сметње између променљивих које припадају **OUT** скупу датих чворова (заокружено црвеном бојом). Чвор 5 не дефинише променљиву и његов **OUT** скуп се игнорише. Последњи, чвор 6 представља **MOVE** инструкцију и примењује се 2. правило, али с обзиром на то да је **OUT** скуп овог чвора празан, такође се не дефинише додатна сметња.



ЧВОР	SUCC	USE	DEF	I		II		III	
				OUT	IN	OUT	IN	OUT	IN
6	-	a	b	-	a	-	a	-	a
5	2, 6	a	-	a	a	ac	ac	ac	ac
4	5	b	a	a	b	ac	bc	ac	bc
3	4	bc	c	b	bc	bc	bc	bc	bc
2	3	a	b	bc	ac	bc	ac	bc	ac
1	2	-	a	ac	c	ac	c	ac	c

Табела 1: Излази из анализе животног века променљивих

	a	b	c
a			X
b			X
c	X	X	

Табела 2: Матрица сметњи

Слика 2: Граф тока управљања

На основу **OUT** скупа последње итерације дефинише се матрица сметњи за сваке две променљиве које се појављују у **OUT** скупу чвора. За дати пример, променљиве [a и c], као и [b и c] имају сметњу што значи да не смеју користити исти регистар.

ПИТАЊЕ: Колико је потребно регистара за реализацију претходног примера?

Додела ресурса – проблем бојења графа

Фазе у компајлеру које претходе фази доделе ресурса подразумевају да на циљаној платформи постоји неограничен број регистара (и ресурса уопште) у које се смештају променљиве. Фаза доделе ресурса има задатак да тај неограничени број регистара сведе на тачно онај број регистара колико има на циљаној платформи: нпр. Mips – 32 регистра. Такође, задатак фазе доделе ресурса је да након доделе регистра открије **MOVE** инструкције које пребацују садржај између истих регистра и обрише такве инструкције. Узмимо, на пример, да је пре почетка фазе доделе ресурса у програму постојала инструкција **A := B**. После фазе доделе ресурса утврђено је да ове променљиве немају никаквих сметњи и обе променљиве A и B имају додељен регистар **R0**. Сада уместо **A := B** у коду стоји инструкција **R0 := R0** коју треба избацити.

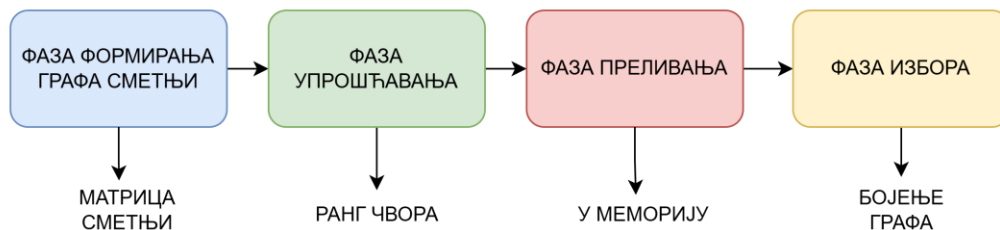
Проблем доделе ресурса се често своди на проблем **бојења графа** применом следећих правила:

1. Једна боја = један регистар
2. Суседни чворови не смеју имати исту боју
3. **ЦИЉ**: Обојити граф са K боја (број расположивих регистра).

Код бојења графа је потребно сваком чвору доделити неку боју, али тако да суседни чворови (чворови који су повезани неком ивицом графа) нису обојени истом бојом. Циљ је обојити произвољни граф са најмањим бројем боја. Ако нам је граф који желимо да бојимо управо граф сметњи, а боје конкретни ресурси циљне архитектуре, онда је повезаност проблема бојења графа и доделе ресурса јасна. Међутим, код доделе ресурса задатак није обојити са што мањим бројем боја (ресурса), већ је довољно обојити са бројем боја мањим од количине слободних ресурса на циљној платформи (на примеру Мипс архитектуре то би било 32). За произвољан граф не постоји ефикасан алгоритам који гарантује оптимално бојење са бројем боја мањим од K (где је K количина расположивих ресурса) у свим случајевима. Ако одређени алгоритам не успе обојити одређени граф са мање од K боја разлог за то може бити једна од две ствари: или граф ни теоретски није могуће обојити са мање од K боја, или јесте могуће обојити теоретски, али конкретан алгоритам не може то да изведе.

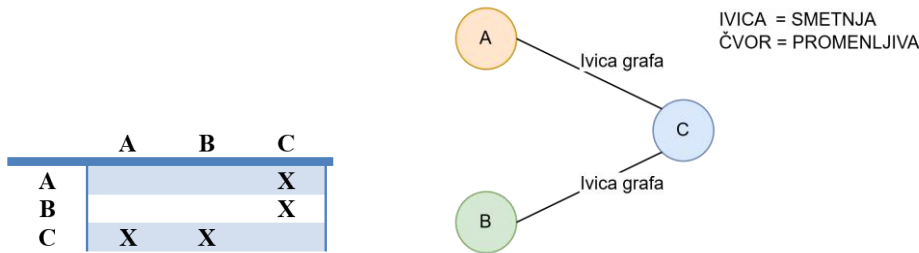
Алгоритам за доделу ресурса који ће у овој вежби бити обрађен састоји се од четири подфазе:

1. ФАЗА ФОРМИРАЊА
2. ФАЗА УПРОШЋАВАЊА
3. ФАЗА ПРЕЛИВАЊА
4. ФАЗА ИЗБОРА



Слика 3: Алгоритам доделе ресурса

Фаза формирања (енгл. *build*): формира граф сметњи на основу анализе животног века променљивих, где свака променљива представља чвор, док су сваке две променљиве повезане ивицом уколико између њих постоји сметња.

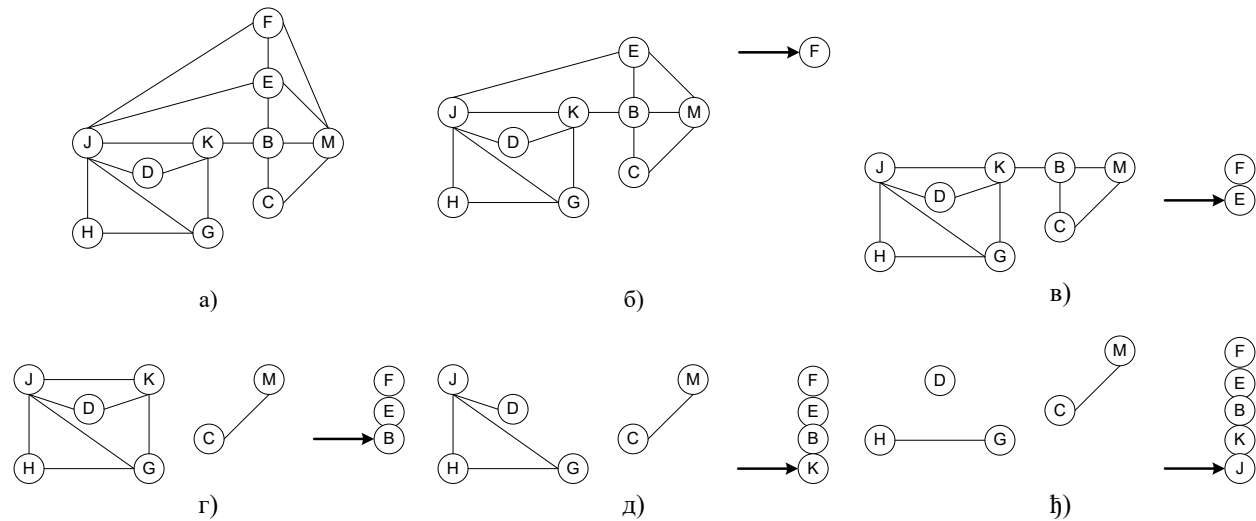


Слика 4: Формирање графа сметњи

Фаза упрошћавања (енгл. *simplify*): граф се боји применом следеће хеуристике: Претпоставимо да чвор М графа G има мање од K суседа. Нека је G' граф који се добија уклањањем чвора М, $G' = G - \{M\}$. Очигледно је да ако је могуће обојити G' онда је могуће обојити и G, пошто кад се чвор М дода обојеном графу G', његови суседи могу бити обојени са највише K-1 боја, тако да увек постоји слободна боја за чвор М. Ово природно води ка рекурзивном алгоритму бојења графа, у ком се чворови чији је ранг мањи од K поступно уклањају из графа сметњи и гурају на стек. Свако упрошћавање доводи до снижавања ранга чвора суседних чвору који се уклања, чиме се ствара могућност за даље упрошћавање.

Ранг чвора представља број његових суседа, односно број чворова са којима је повезан.

На следећој слици дат је пример упрошћавања графа сметњи за процесор који има 4 регистра, дакле K је 4 (а). Редом се бирају чворови који се уклањају из графа. Најпре се бира чвор са највећим рангом мањим од K. На датом примеру графа, чвор F има највећи ранг мањи од 4 (3), зато се ставља на стек и том приликом се брише са графа. Брисање подразумева брисање самог чвора и брисање свих веза тог чвора са његовим суседима (б). Затим се открива да чвор E има ранг 3, исто мањи од 4, тако да се он брише са графа и ставља на стек (в). Остатак примера се лако може испратити. Фаза упрошћавања се завршава када се обришу сви чворови са графа и поставе на стек. Чворови се касније морају бојити обрнутим редоследом у односу на то како су уклањани и због тога је стек природна структура података за ову фазу.



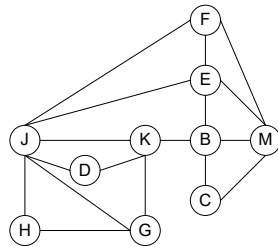
Слика 4: Пример упрошћавања графа сметњи

Пример распореда чворова на стеку: F, E, B, K, J, M, H, D, C, G

Фаза преливања (енгл. *spill*): у току фазе упрошћавања може се десити да се граф сметњи не може више упростити, а да још увек постоје чворови у њему. То значи да се дати граф не може обојити са K боја. Тада се прибегава растерећивању графа тако што ће неким променљивама бити промењен ресурс, то јест биће пребачене из регистара у, на пример, меморију. Тај поступак се назива преливање, сликовито описујући шта се дешава са преосталим променљивама када је скуп ресурса препуњен – преливају се у меморију. Након овога обично се мора поновити фаза избора инструкција над целим или делом међукода, па затим опет анализа животног века, и на крају поново овај алгоритам доделе ресурса. Оваква петља се може извршити више пута, уколико је граф сметњи превише сложен. Ова фаза успорава програм, али омогућава наставак доделе ресурса.

Фаза избора (енгл. *select*): након што је граф успешно упрошћен потребно је обојити чворове. Редослед бојења чворова је одређен стеком на који су се чворови постављали приликом упрошћавања графа. Дакле, чворови ће бити бојени **обрнутим редоследом** од оног којим су скидани са графа. Једино правило које се користи приликом бојења је да се **чвору не сме доделити боја која је већ додељена неком од суседних чворова**.

Пошто су чворови на стек постављени редом F, E, B, K, J, M, H, D, C, G, онда се у фази избора боје додељују обрнутим редом G, C, D, H, M, J, K, B, E, F и резултат би могао да изгледа овако:



Чвор на стеку	Регистар
G	R0
C	
D	
H	R1
M	
J	R2
K	R1
B	R2
E	R0
F	R3



НАПОМЕНА: Бојење графа није јединствено тј. различити редоследи избора слободног регистра могу дати различите, али исправне расподеле ресурса.

Реализација

Приликом реализације задатака потребно је користити постојеће структуре и функције:

- **Типови података** описани су у датотеци *Types.h*.
- **Инструкције** су описане у структури *Instruction*.

У оквиру ове вежбе користи се: тип инструкције (*type*), показивачи на одредишни, први и други операнд (*dst*, *src1*, *src2*). Више инструкција описује се помоћу листе у типу *Instructions*.

```
typedef struct InstructionStruct
{
    InstructionType type;

    Variable* dst;
    Variable* src1;
    Variable* src2;
} Instruction;

typedef list<Instruction*> Instructions;
```

- **Променљиве** су описане у структури *Variable* и садрже:
 - назив (*name*),
 - позицију унутар матрице сметњи (*pos*),
 - боју ресурса (*assignment*).

Више променљивих описано је помоћу листе у типу *Variables*.

```
struct Variable
{
    char name;
    int pos;
    Regs assignment;
};

typedef list<Variable*> Variables;
```

- **Типови инструкција** описани су преко набрајајућег типа *InstructionType*, и подржане су следеће инструкције: *T_MOVE*, *T_EQU*, *T_SUB*, *T_ADD*, *T_MUL*, *T_COND*, *T_RET* и *T_OTHER*.
- **Боје ресурса** описане су у типу *Regs* и подржани су регистри *reg0*, *reg1*, *reg2* и *reg3*.
- **Граф сметњи** описан је у структури *InterferenceGraph* која садржи листу свих променљивих коришћених у инструкцијама (*variables*), матрицу сметњи (*values*) и величину матрице (*size*). Матрица сметњи описана је преко двоструког низа који садржи вредност `__INTERFERENCE__` на местима где постоји сметња између променљивих и `__NO_INTERFERENCE__` на местима где нема сметњи.

```
typedef struct
{
    Variables* variables;
    char** values;
    int size;
} InterferenceGraph;
```

За пример се може узети матрица сметњи из Табеле 2 која садржи 3 променљиве А, В и С. Променљива А има позицију 0, В позицију 1 и С позицију 2. Позиције (**pos** из структуре **Variable**) треба искористити за правилно индексирање матрице сметњи. Променљиве у листи *variables*, из структуре *InterferenceGraph*, не морају бити увезане оним редом којим се налазе у матрици сметњи. Нпр. у листи се редом налазе С, А и В, док матрица сметњи (*values*) описује редом сметње између променљивих А, В и С.

Приликом решавања задатка је потребно користити већ постојеће статичке библиотеке: *libResourceAllocation.lib* и *libLivenessAnalysis.lib*.

За рад са инструкцијама потребно је користити функције из датотеке *Instructions.h*. Ту се налазе функције за испис једне или више инструкција у конзолу. Ове функције треба користити приликом откривања грешака у програму, као и за приказ резултата програма.

```
void printInstruction(Instruction* instr);  
void printInstructions(Instructions* instrs);
```

За рад са променљивама је потребно користити функције из датотеке *Variable.h*. Ту се налазе функције за испис једне или више променљивих у конзолу. Ове функције треба користити приликом откривања грешака у програму, као и за приказ резултата програма.

```
void printVariables(Variables* variables);  
void printVariable(Variable* variable);
```

Пример међукода за који се после ради анализа животног века променљивих, те додела ресурса, може се направити позивом функције *makeExample* која враћа листу припремљених инструкција.

Реализација **анализе животног века** променљивих налази се библиотеци *libLivenessAnalysis.lib* и датотеци *LivenessAnalissysLib.h*. Позивом функције *doLivenessAnalysis* за прослеђену листу инструкција се добија нова листа инструкција са анализом животног века променљивих. Уколико се преко аргумента *debug* пренесе вредност `__DEBUG__` исписаће се додатне информације везане за анализу животног века променљивих.

```
Instructions* doLivenessAnalysis(Instructions* instructions, int debug = __NO_DEBUG_);
```

За рад са **графом сметњи** је потребно користити функције из датотеке *InterferenceGraph.h*. Помоћу функције *doInterferenceGraph* од листе инструкција се прави граф сметњи, тј. структура *InterferenceGraph* (прва фаза доделе ресурса). Том приликом се заузима меморија за двоструки низ (матрица сметњи) коју је потребно ослободити на крају програма позивом функције *freeInterferenceGraph*. За испис графа сметњи у конзолу треба користити функцију *printInterferenceGraph*.

```
InterferenceGraph* doInterferenceGraph(Instructions* instructions);  
void freeInterferenceGraph(InterferenceGraph* ig);  
void printInterferenceGraph(InterferenceGraph* ig);
```

Фаза упрошћавања је реализована у библиотеци *libResourceAllocation.lib* у датотеци *Simplification.h*, и функцији *doSimplification*. Она као аргументе прихвата граф сметњи и степен упрошћавања. Као повратна вредност функције, враћа се попуњен стек (Слика 2). Уколико је показивач на стек празан, тада фаза упрошћавања није успела да упрости граф сметњи за жељени степен, тј. дошло је до преливања.

```
stack<Variable*>* doSimplification(InterferenceGraph* ig, int degree);
```

ЗАДАТАК

На основу прослеђеног стека и графа сметњи (имплементиран део), реализовати последњу фазу доделе ресурса – **фазу избора** која се налази у функцији `doResourceAllocation` у датотеци *ResourceAllocation.h*. Она као аргументе очекује стек из фазе упрошћавања и показивач на граф сметњи. Уколико се успешно доделе ресурси свим променљивама, ова функција враћа тачну вредност и обрнуто.

```
bool doResourceAllocation(stack<Variable*>* simplificationStack,  
InterferenceGraph* ig);
```

КОРАЦИ реализоване фазе доделе ресурса:

- Преузети чвор са врха стека (`top()` и `pop()`),
- Доделити боје:
 - Ако је преузета прва променљива, доделити јој прву слободну боју тј. `reg`.
 - Уколико није прва променљива, проверити да ли је у сметњи са претходно обојеном и доделити јој нову боју (ако постоји сметња).
 - Уколико сметња не постоји, променљива се може обојити истом бојом као и претходна.
- Поновити поступак док стек не постане празан.

ЗАДАТАК: Реализовати функцију за доделу боје:

```
int getColor(Variable* notColoredVariable, InterferenceGraph* ig)
```

Алгоритам најпре редом пролази кроз све доступне регистре. За сваки регистар потребно је проверити да ли га већ користи нека променљива која је у сметњи са посматраном променљивом.

Провера сметње врши се помоћу матрице сметњи:

```
ig->values[pos1][pos2]
```

Ако постоји сметња и нека суседна променљива већ користи тренутни регистар, тај регистар није дозвољен.

Уколико се пронађе регистар који не користи ниједан суседни чвор, тај регистар се додељује посматраној променљивој.

Ако ниједан регистар није слободан, функција враћа вредност `__UNDEFINE__`.

ДОДАТНИ ЗАДАТАК

Реализовати брисање `T_MOVE` инструкција у оквиру функције `removeMove` у датотеци *ResourceAllocation.cpp*. Ова функција, након завршетка доделе ресурса, елиминише све инструкције које представљају доделу регистра самом себи.

```
Instructions * removeMove(Instructions * instrs);
```